BROTAR

Principios, patrones y defensa técnica
Aplicación en el Sprint 2

Informe académico para docente y equipo. Ricardo, Alison y Santiago. Corte local 05-10-2026, rama RicardoDev. Se agrupan los ejemplos por concepto para localizar qué se usó, dónde, por qué y cómo defenderlo.

Tesis del informe

La calidad se demuestra mediante decisiones concretas, contratos y pruebas, no con una lista de siglas. Se conservó la arquitectura del Sprint 1 y se refactorizó únicamente donde hubo un beneficio verificable. Un patrón no se añade solo para cumplir una rúbrica.

Qué es cada cosa

TypeScript es el lenguaje. POO, funciones puras, interfaz declarativa y asincronía son formas de programación utilizadas. SOLID, KISS, DRY, YAGNI y Boy Scout orientan diseño/mantenimiento. Repository y Adapter son patrones; transacciones son mecanismos de consistencia. Scrum/sprints organizan el trabajo, no son patrones de código.

Organización del documento

Arquitectura y paradigmas; KISS; DRY; YAGNI; S, O, L, I y D de SOLID; IoC/DI y composición; Boy Scout; patrones Repository/Adapter y consistencia transaccional; validación/seguridad; pruebas, defensa y referencias.

Cautelas

Los ejemplos se sostienen en código local y pruebas. Los anclajes de línea se verifican al generar; buscar el símbolo si luego cambia el archivo. No se afirma que todo Brotar sea SOLID perfecto ni que un cambio de base o una nueva regla sea gratis. No hay commit/push de este corte ni aceptación externa.

276 pruebas aprobadas (130 frontend, 128 backend, 18 PostgreSQL). Incluyen regresión de Sprint 1; no equivalen a historias cerradas ni a una certificación absoluta de seguridad.

01. Arquitectura y paradigmas en uso

Unidad	Responsabilidad	No debe decidir
Vista React	Controles, mensajes, acciones visibles	Autorización definitiva / SQL
Hook / cliente	Estado, solicitudes y contratos	Roles confiados desde formulario
Controlador HTTP	DTO, sesión, rol, respuesta	Texto SQL del caso de uso
Aplicación	Coordinar operación mediante puertos	DDL / detalles de Express
Dominio	Reglas puras y errores del negocio	Conexión de red o base
Adaptador TypeORM	Consultas y transacciones	Reglas comerciales inventadas

Programación orientada a objetos

Casos como AdministrativeReview y adaptadores reúnen operaciones y colaboradores en clases. No se usa POO por obligación en cada función ni se construyen jerarquías de entidades para representar roles.

Funciones y declaración de interfaz

validatePlan, validateFunding y missingForSubmission reciben datos y producen resultados sin red. React declara la interfaz a partir de estado/props. No se afirma programación funcional estricta: el proyecto combina estilos y efectos explícitos.

Asincronía y consistencia

async/await espera la API y la base sin bloquear la interfaz. Promise.all carga secciones independientes. Las lecturas administrativas coherentes usan el mismo manager secuencialmente y REPEATABLE READ. Asincronía por sí sola no garantiza atomicidad ni evita carreras.

backend/src/campaigns/infrastructure/typeorm-administrative.repository.ts:20
Buscar: transaction('REPEATABLE READ'

backend/src/campaigns/domain/submission.ts:28
Buscar: export function missingForSubmission

Ejemplo para seguir en la defensa: CampaignSectionsEditor > useCampaignSections > sectionsClient > SectionsController > CampaignSections > TypeormSectionsRepository > PostgreSQL. Una respuesta vuelve por contratos y estado antes de anunciar éxito.

02. KISS - simplicidad verificable

Concepto y problema que resuelve

Elegir la solución más simple que satisfaga el requisito, no la más corta a costa de seguridad. En este sprint las decisiones se expresan mediante funciones de dominio pequeñas y operaciones explícitas; no se añadió un motor genérico de campañas.

Dónde y por qué se aplica en Brotar

reviewTarget, isPublicCampaign y canDiscardCampaign conservan reglas concretas. validateFunding revisa meta/modelo/duración/fechas y devuelve errores por campo. La corrección decimal compara con toFixed(2) en lugar de una tolerancia binaria arbitraria; budgetTotal suma céntimos dentro del límite actual de partidas.

Ubicación exacta para abrir durante la defensa

backend/src/campaigns/domain/lifecycle.ts:6
Buscar: export function reviewTarget

backend/src/campaigns/domain/sections.ts:19
Buscar: const money

src/features/campaigns/sectionsClient.ts:98
Buscar: export function budgetTotal

Cómo demostrar el beneficio

Mostrar una campaña APPROVED que aún no aparece públicamente; abrir isPublicCampaign. Ejecutar el caso 0.10 + 0.20 y el importe máximo válido. La regla es legible y se puede aislar en una prueba sin arrancar NestJS.

Límite y afirmación que debe evitarse

Una función pequeña puede seguir siendo incorrecta. Sumar céntimos aquí no constituye un motor financiero listo para pagos: es un total de presentación sobre partidas validadas y límites acotados.

Respuesta breve al docente

“Aplicamos KISS a reglas acotadas y resultados explícitos; mantuvimos transacciones y autorización porque forman parte del requisito, no son complejidad prescindible”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

03. DRY - reutilizar conocimiento, no acoplar fronteras

Concepto y problema que resuelve

Evitar copias de una misma decisión que evolucionen de forma contradictoria. Se conserva transporte HTTP común y parsers reutilizados. No se elimina la validación de servidor por existir una en la interfaz.

Dónde y por qué se aplica en Brotar

requestApi centraliza cookies, no-store, timeout, cabecera de mutación y traducción de fallos. administrativeDetailFrom reutiliza planFrom, budgetFrom, fundingFrom y rewardsFrom. El hook concentra el patrón de guardado y feedback para cuatro bloques sin copiarlo en cada formulario.

Ubicación exacta para abrir durante la defensa

src/shared/api/request.ts:11
Buscar: export async function requestApi

src/features/campaigns/administrativeClient.ts:25
Buscar: export function administrativeDetailFrom

src/features/campaigns/useCampaignSections.ts:46
Buscar: async function persist

Cómo demostrar el beneficio

Abrir tests/api-request.test.ts y observar cookies/cancelación. Abrir administrative-campaigns.test.ts: una respuesta anidada incorrecta no crea un éxito falso. Mostrar cómo cada formulario invoca una operación del mismo hook.

Límite y afirmación que debe evitarse

Frontend y backend tienen reglas espejo de secciones porque protegen fronteras distintas; son duplicación consciente, no DRY perfecto. Pruebas de paridad evitan divergencia en los casos cubiertos. El navegador no importa código de infraestructura del backend. Cambiar la regla exige revisar ambos lados.

Respuesta breve al docente

“Reutilizamos transporte y contratos; el servidor sigue siendo autoridad. No confundimos eliminar copia con eliminar una defensa independiente”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

04. YAGNI - alcance confirmado, no funciones supuestas

Concepto y problema que resuelve

No construir hoy funcionalidades basadas solo en una posibilidad futura. La tarea Fase 3 excluye explícitamente pagos, aportes, checkout, KYC/KYB y rendición. El backlog conserva esos módulos para refinamiento posterior.

Dónde y por qué se aplica en Brotar

PublicCampaigns expone catálogo/detalle; Projects controla proyectos/descarte; AdministrativeReview decide y publica según D05. No se añadió un proveedor de pagos, permisos definitivos inventados ni activación de identidad automática. La configuración se limita a publicación administrativa provisional.

Ubicación exacta para abrir durante la defensa

backend/src/campaigns/application/public-campaigns.ts:12
Buscar: export class PublicCampaigns

backend/src/campaigns/application/projects.ts:23
Buscar: export class Projects

backend/src/campaigns/domain/administrative-review.ts:29
Buscar: export function publicationEnabled

Cómo demostrar el beneficio

En el detalle público mostrar contenido/recompensas sin cobrar. Explicar que configurar una meta y mostrar avance real no implementa checkout. Abrir ADR-005 y señalar alcance provisional y decisión D06 pendiente.

Límite y afirmación que debe evitarse

YAGNI no justifica omitir pruebas, seguridad, documentación o publicación requerida por la fase. Tampoco significa borrar el backlog futuro. El diseño deja puertos acotados, no una plataforma extensible para cualquier caso imaginado.

Respuesta breve al docente

“Construimos el flujo real pedido y no adelantamos los módulos expresamente excluidos. Las decisiones del cliente pendientes se documentan en vez de inventarse”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

05. S de SOLID - responsabilidad única (SRP)

Concepto y problema que resuelve

Una unidad debe concentrar una responsabilidad y una razón coherente para cambiar. Separar archivos por sí solo no basta: cada pieza debe conservar un propósito claro.

Dónde y por qué se aplica en Brotar

Se extrajo useCampaignSections del componente. El hook coordina carga, guardado, cambios pendientes y errores; CampaignSectionsEditor describe controles. sectionsClient transporta y valida contratos. En backend SectionsController recibe HTTP, CampaignSections coordina el caso y TypeormSectionsRepository realiza persistencia.

Ubicación exacta para abrir durante la defensa

src/features/campaigns/useCampaignSections.ts:9
Buscar: export function useCampaignSections

src/features/campaigns/CampaignSectionsEditor.tsx:17
Buscar: export function CampaignSectionsEditor

backend/src/campaigns/application/sections.ts:27
Buscar: export class CampaignSections

Cómo demostrar el beneficio

Buscar en el editor: no contiene fetch ni las operaciones de persistencia. Mostrar en el hook persist y sus callbacks de guardado. Ejecutar el render SSR de carga y las pruebas de cliente/reglas; el constructor continúa integrando los bloques.

Límite y afirmación que debe evitarse

El hook coordina cuatro bloques relacionados y el constructor general conserva bastante estado. No se afirma que todo archivo tenga una única tarea microscópica ni que esta extracción reduzca por sí sola el número de bugs. Una prueba interactiva adicional del hook no fue repetida por bloqueo del navegador.

Respuesta breve al docente

“Separé coordinación y presentación para poder cambiar mensajes/controles sin reescribir red, y cambiar guardado sin alterar todo el marcado”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

06. O de SOLID - abierto/cerrado (OCP)

Concepto y problema que resuelve

Diseñar puntos de variación que permitan extender comportamientos sin reescribir consumidores estables. No exige que nunca se modifique una línea existente: una corrección o un nuevo requisito puede requerir cambios.

Dónde y por qué se aplica en Brotar

AdministrativeReview depende del contrato AdministrativeRepository. La misma coordinación se prueba con un repositorio en memoria y se ejecuta con TypeormAdministrativeRepository. requestApi admite un transporte send y una estrategia errorFromResponse; los módulos concretos validan sus propios contratos.

Ubicación exacta para abrir durante la defensa

backend/src/campaigns/application/administrative-review.ts:18
Buscar: export interface AdministrativeRepository

backend/src/campaigns/application/administrative-review.ts:23
Buscar: export class AdministrativeReview

src/shared/api/request.ts:7
Buscar: errorFromResponse?

Cómo demostrar el beneficio

Mostrar en administrative-review.test.ts el objeto repository que cuenta escrituras. Activar/desactivar publicación sin cambiar el caso. Mostrar cómo el cliente de campañas especializa errores 400 sin duplicar todo requestApi.

Límite y afirmación que debe evitarse

Cambiar PostgreSQL por otro motor sigue exigiendo implementar consultas, restricciones y migraciones; no es una sustitución gratuita. Agregar un nuevo estado o decisión podría modificar dominio, contratos, UI y base: no se anuncia OCP universal.

Respuesta breve al docente

“Dejamos extensible la persistencia y el transporte en puntos concretos; no añadimos una jerarquía por cada modalidad solo para mostrar el principio”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

07. L de SOLID - sustitución de Liskov (LSP)

Concepto y problema que resuelve

Una implementación o especialización debe conservar el contrato esperado por quien la consume. Coincidir en nombres o tipos no basta si cambia las promesas de errores, resultados o invariantes.

Dónde y por qué se aplica en Brotar

DraftFieldError extiende SessionError, mantiene status=400 y agrega fields. requestApi conserva errores que ya son SessionError, de modo que vistas generales pueden tratarlos sin perder su estado HTTP; vistas de formulario aprovechan el detalle. Los dobles de repositorio conservan promesas y resultados del puerto.

Ubicación exacta para abrir durante la defensa

src/features/campaigns/campaignsClient.ts:50
Buscar: export class DraftFieldError

src/shared/api/sessionError.ts:2
Buscar: export class SessionError

tests/api-request.test.ts:22
Buscar: test('LSP

Cómo demostrar el beneficio

Ejecutar el caso LSP de api-request.test.ts: el error especializado es también SessionError y conserva status. Después probar un formulario inválido: se muestran campos, sin convertir el error en un éxito genérico.

Límite y afirmación que debe evitarse

Un fake en memoria no demuestra por sí solo todas las garantías del repositorio SQL, especialmente concurrencia. La suite PostgreSQL complementa esa evidencia. No se creó herencia artificial entre usuario, creador y organización: esas responsabilidades se expresan mediante roles/relaciones.

Respuesta breve al docente

“El error más específico puede utilizarse donde se esperaba SessionError sin romper el consumidor; añade detalle, no cambia su significado”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

08. I de SOLID - segregación de interfaces (ISP)

Concepto y problema que resuelve

Los consumidores no deberían depender de operaciones que no necesitan. Un contrato se define por su uso, no por intentar reunir todo el sistema en una interfaz universal.

Dónde y por qué se aplica en Brotar

SubmissionRepository ofrece snapshot, submit y observations. AdministrativeRepository se limita a detail, decide y publish. PublicCampaignRepository solo lista el contenido público. La capa de casos de uso no exige métodos de pagos, KYC o perfil para consultar campañas.

Ubicación exacta para abrir durante la defensa

backend/src/campaigns/application/submission.ts:8
Buscar: export interface SubmissionRepository

backend/src/campaigns/application/administrative-review.ts:18
Buscar: export interface AdministrativeRepository

backend/src/campaigns/application/public-campaigns.ts:10
Buscar: export interface PublicCampaignRepository

Cómo demostrar el beneficio

Abrir los tres contratos y comparar operaciones. Mostrar un doble de AdministrativeRepository que implementa únicamente tres métodos para ensayar decisiones. Mostrar que el catálogo no recibe datos privados de revisión.

Límite y afirmación que debe evitarse

SectionsRepository conserva varios métodos de lectura/escritura y catálogo porque el constructor los utiliza. Podría refinarse si aparecen consumidores con necesidades distintas, pero no se crearon decenas de interfaces sin necesidad actual. Interfaces pequeñas no garantizan por sí mismas privacidad.

Respuesta breve al docente

“Los puertos se separan por caso de uso; el catálogo no debe implementar aprobación, autenticación ni pagos”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

09. D de SOLID - inversión de dependencias (DIP)

Concepto y problema que resuelve

Las reglas de alto nivel dependen de abstracciones de su propia capa, no de detalles de HTTP o persistencia. La inyección es una técnica de ensamblaje; no equivale automáticamente al principio.

Dónde y por qué se aplica en Brotar

CampaignSections recibe DraftRepository y SectionsRepository; CampaignSubmission recibe puertos de borradores/envío; AdministrativeReview recibe su repositorio y una política booleana. Dominio/aplicación no importan NestJS, Express, TypeORM ni infrastructure. campaigns.module.ts ensambla adaptadores concretos.

Ubicación exacta para abrir durante la defensa

backend/src/campaigns/application/sections.ts:6
Buscar: constructor(

backend/src/campaigns/campaigns.module.ts:43
Buscar: provide: AdministrativeReview

backend/test/architecture.test.ts:6
Buscar: test('BG-58

Cómo demostrar el beneficio

Ejecutar architecture.test.ts, que inspecciona imports de capas. Luego mostrar una prueba de aplicación que inyecta un repositorio fake. Finalmente seguir la factory de Nest para explicar qué implementación se usa realmente.

Límite y afirmación que debe evitarse

La prueba de imports detecta una forma de acoplamiento, no garantiza una arquitectura perfecta. El adaptador debe conocer TypeORM y PostgreSQL: eso corresponde a infraestructura. Al cambiar el esquema se revisan adaptadores y migraciones, no solo el contenedor.

Respuesta breve al docente

“Los casos de uso piden operaciones de negocio mediante puertos; el módulo decide cómo conectarlos con PostgreSQL”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

10. IoC, inyección y composición sobre herencia

Concepto y problema que resuelve

IoC traslada el control de ensamblar dependencias fuera del caso de uso. Dependency Injection entrega esos objetos desde el exterior. Composición reúne colaboradores sin hacer que una clase herede de otra con una responsabilidad distinta.

Dónde y por qué se aplica en Brotar

Nest registra adaptadores y useFactory construye los casos. CampaignSections usa repositorios; AdministrativeReview usa uno y una bandera. En frontend CampaignBuilderPage compone editor, portada y revisión; el editor utiliza un hook. No hereda de un “formulario universal”.

Ubicación exacta para abrir durante la defensa

backend/src/campaigns/campaigns.module.ts:39
Buscar: provide: CampaignSections

src/features/campaigns/CampaignBuilderPage.tsx:9
Buscar: CampaignSectionsEditor

src/features/campaigns/CampaignSectionsEditor.tsx:20
Buscar: useCampaignSections(props)

Cómo demostrar el beneficio

Seguir el ensamblaje: proveedor concreto > constructor > operación. Cambiar en un test el colaborador inyectado y observar la misma coordinación. Mostrar que actualizar portada no obliga a crear subclases de los demás pasos.

Límite y afirmación que debe evitarse

La herencia no está prohibida: DraftFieldError es una especialización válida y algunos DTO comparten campos. Composición se prefiere cuando la relación es “usa/tiene”, no “es un”. Estos conceptos son distintos de Repository y de SOLID, aunque se complementen.

Respuesta breve al docente

“No hacemos que la revisión sea una base de datos; la revisión usa un repositorio que recibe al construirse”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

11. Boy Scout Rule - mejoras acotadas con regresión

Concepto y problema que resuelve

Dejar el área intervenida en mejor estado mediante correcciones pequeñas y justificadas. No convertir una entrega en una reescritura completa ni tocar código correcto para aparentar más trabajo.

Dónde y por qué se aplica en Brotar

En esta revisión se corrigieron la comparación decimal, suma de céntimos, fechas imposibles, NUL en texto persistido, límites técnicos de 5000, respuesta de ronda inválida y límite HTML de contraseñas Unicode. Se añadió edición de notas de partida y se separó el hook; se mantuvieron hash, sesión y arquitectura existentes.

Ubicación exacta para abrir durante la defensa

backend/test/sprint2-quality.test.ts:15
Buscar: test('S2 calidad

tests/sprint2-quality.test.tsx:18
Buscar: test('S2 calidad

src/features/access/AccessComponents.tsx:18
Buscar: const htmlLimit

Cómo demostrar el beneficio

Mostrar antes/problema/después con un caso reproducible: importe válido alto, 31/02, nota NUL o contraseña con emoji. Ejecutar la regresión y explicar qué decisión no cambió. Las notas de presupuesto ya existían en el contrato; se hicieron editables.

Límite y afirmación que debe evitarse

5000 es límite técnico de entrada, no aprobación del cliente sobre el tamaño del contenido. No se prometen validaciones perfectas para toda futura funcionalidad ni seguridad absoluta. El límite de contraseñas HTML permite hasta dos unidades UTF-16 por punto; la validación sigue rechazando más de 128 puntos de código.

Respuesta breve al docente

“Mejoramos fallos observables y su evidencia sin rehacer el proyecto ni inventar reglas pendientes”.

Distinguir conservación y cambio: contratos/capas/transporte ya existían; la extracción del hook y las correcciones de validación se efectuaron en esta revisión. Una guía de principios describe ejemplos representativos, no una certificación de todo el repositorio.

12. Patrones adecuados y consistencia

Repository: intención de negocio frente a persistencia

AdministrativeRepository, SubmissionRepository y PublicCampaignRepository definen operaciones sin SQL. Los adaptadores TypeORM implementan consultas, bloqueos y relaciones. Beneficio: probar coordinación con dobles y concentrar cambios del esquema. TypeORM no sustituye los contratos de negocio; aquí se utilizan además consultas SQL explícitas.

backend/src/campaigns/application/administrative-review.ts:18
Buscar: export interface AdministrativeRepository

backend/src/campaigns/infrastructure/typeorm-administrative.repository.ts:14
Buscar: export class TypeormAdministrativeRepository

Adapter: traducir contratos

Los repositorios traducen puertos a PostgreSQL; mapPublicCampaign traduce el DTO público a Project usado por tarjetas/detalle, verificando tipos, identidad, importes e imagen. Un 404 o contrato inválido no se rellena con una campaña mock. El adaptador reduce impacto entre formatos distintos.

src/features/public/campaignsApi.ts:23
Buscar: export function mapPublicCampaign

Transacción y concurrencia: mecanismo, no etiqueta decorativa

Una decisión actualiza revisión, observaciones, campaña e historial dentro de una transacción. SELECT FOR UPDATE bloquea campaña y última ronda; expectedRound detecta vista obsoleta. Publicar vuelve a exigir APPROVED y completitud. El trigger oficial produce historial; el repositorio lo enriquece sin duplicarlo.

backend/src/campaigns/infrastructure/typeorm-administrative.repository.ts:51
Buscar: private async lock

Qué no se presenta como patrón implementado

No se afirma Event Sourcing, CQRS completo, máquina de estados genérica ni Unit of Work propio. TypeORM aporta transacciones; separar lecturas/escrituras no prueba por sí solo CQRS. Los parámetros/ports facilitan estrategias de prueba, sin exigir una jerarquía GoF Strategy innecesaria.

Prueba de beneficio: dos solicitudes concurrentes producen una transición y un conflicto, no dos rondas/decisiones. Si se desactiva un catálogo antes de aprobar, se rechaza y conserva la revisión pendiente.

13. Validaciones y protección verificadas

Dato / acción	Comportamiento sustentado
Nombres personales	Unicode de letras/marcas, separadores permitidos; rechaza números/símbolos no admitidos
Títulos / descripciones	Aceptan Ñ, acentos y números; límites por campo; NUL rechazado antes de persistir
Contraseña real	15-128 puntos de código, no solo espacios; coincidencia exacta; no trim ni normalización
Importes / fechas	Positivos, finitos, numeric(14,2), días reales; fecha final coherente
Escrituras privadas	Sesión, rol/propiedad, origen/cabecera, UUID/DTO; bloqueos/transacciones
Datos públicos	Solo PUBLISHED con imagen pública válida; sin respaldo mock ante error

Contraseña y sesión: protección existente conservada

Scrypt usa sal aleatoria de 16 bytes, N=16384, r=8, p=5 y comparación timingSafeEqual. No se guarda contraseña clara. La sesión es un token opaco aleatorio; su hash queda en la base. Cookie HttpOnly/SameSite Strict; Secure en producción. Caducidad, revocación, límites de solicitudes y origen se verifican en las suites existentes.

backend/src/users/infrastructure/scrypt-password-hasher.ts:7
Buscar: export class ScryptPasswordHasher

backend/src/auth/infrastructure/http/session-http.ts:12
Buscar: export function sessionCookieOptions

Unicode no es “corregir la contraseña”

“Ñ” no se trata como “N”; “é” y “e + acento combinante” no se convierten en la misma contraseña. Los espacios iniciales/finales forman parte del secreto. El hash y el login exactos se probaron. La búsqueda de campañas sí normaliza acentos por conveniencia: no se reutiliza esa normalización para credenciales.

Límites de la protección

El medidor de registro informa longitud, no entropía completa. Validación no sustituye autorización y parametrizar SQL no elimina todas las amenazas. Producción necesita HTTPS, secretos/configuración, operación y políticas aprobadas; esta entrega local no se declara auditada integralmente.

14. Evidencia, defensa y referencias

Mapa de pruebas

Funciones puras: campañas/secciones/estados y límites. Aplicación: repositorios fake, escritura denegada y bandera. HTTP: DTO anidados, sesión, roles, origen, campos extra y errores. Frontend: transporte, parsers, SSR y paridad de reglas. PostgreSQL: flujo real, rollback, persistencia, carreras, rondas/publicación y migración aislada.

backend/test/architecture.test.ts:6
Buscar: test('BG-58

backend/test/sprint2-quality.test.ts:15
Buscar: test('S2 calidad

tests/sprint2-quality.test.tsx:18
Buscar: test('S2 calidad

Ensayo de defensa: problema > decisión > código > prueba

Ejemplo SRP: formulario mezclaba coordinación y marcado > extraer hook > abrir ambos archivos > demostrar carga y guardado. Ejemplo DIP: caso no debe conocer TypeORM > puerto > abrir constructor/factory > prueba con fake. Ejemplo validación: NUL podía provocar fallo de base > rechazo por campo > abrir regla > prueba y conservación de datos.

Preguntas que deben poder contestar

¿Por qué no se reemplazó fetch por Axios? El transporte común ya satisface el alcance y tiene pruebas; la biblioteca no determina el paradigma ni la calidad. ¿Dónde se ve ISP? Puertos de revisión/envío/catálogo. ¿Dónde se ve LSP? DraftFieldError mantiene status de SessionError. ¿Cambiar de base sería automático? No: se revisan adaptadores, esquema, restricciones y migraciones.

Fuentes conceptuales suministradas

SOLID.pdf, DRY.pdf y YAGNI.pdf, Lic. M.Sc. Benjamín H. Buitrago Conde: definiciones y ejemplos de los principios. APUNTES DE PROYECTO DE SISTEMAS II Teams.docx, unidad I, 1.2 y otros principios: KISS, IoC, Boy Scout y composición; apartados de pruebas. Se adaptan los conceptos, no se copia el stack C# del ejemplo.

Fuentes del producto y alcance

Tarea Fase 3, módulo de campañas: requisitos, validaciones y exclusiones; README y planificación local; ADR-005; código/tests de los anclajes. La rúbrica suministrada para Sprint 1 orienta la defensa de calidad/SOLID/patrones; no se inventa una nueva ponderación para Sprint 2.

Cierre: evidencia técnica local, no aprobación externa ni garantía de ausencia de fallos. La guía de demostración compañera desarrolla las trece tareas y sus checklists. Preparar cuentas autorizadas ficticias; no exponer tokens, .env o datos personales en la presentación.